

❏ What is Exploit Development?

Exploit development is the process of writing custom code to take advantage of vulnerabilities in a software or system to:

- Gain unauthorized access
- Execute arbitrary code
- Elevate privileges
- Disrupt services (DoS)

This is a core aspect of **ethical hacking**, **penetration testing**, and **malware analysis**.

❏ Phases of Exploit Development

1. Reconnaissance

- Identify version, system, architecture.
- Use tools like nmap, nikto, dirb, whatweb.

2. Vulnerability Identification

- Look for known CVEs or 0-days.
- Static/dynamic analysis.

3. Fuzzing

- Sending unexpected/malformed inputs to find crashes.
-

Tools: Peach, Boofuzz, AFL.

4. Debugging

- Attach debugger (e.g., GDB, OllyDbg) to monitor crashes and flow.

5. Buffer Overflow

- Overwrite memory to control execution.
- Use of:
 - NOP sleds
 - Shellcode
 -

Return Address Overwrite

6. Return Oriented Programming (ROP)

- Bypass modern protections (ASLR, DEP) by chaining gadgets already present in memory.

7. Payload Crafting

- Write shellcode or use frameworks like msfvenom.

8. Post Exploitation

- Maintain access (e.g., reverse shells).
- Privilege escalation.

- Cover tracks.

☒ **Protections to Bypass**

- **ASLR:** Address Space Layout Randomization
- **DEP/NX:** Data Execution Prevention
- **Stack Canaries**
- **PIE:** Position Independent Executables

☒ **Part 2: Writing Plugins in Python**

❏ What is a Plugin?

A **plugin** is a piece of code that extends the functionality of an application **without modifying its core logic**.

❏ Benefits of Plugin Architecture

- Modular
 - Extensible
 - Easier maintenance
 - Code reuse
-

☒ Plugin Use Cases

- Add custom features to existing tools (e.g., Burp Suite extensions)
 - Automate scans or reporting in security tools
 - Integrate external APIs or modules
-

☒ Plugin Architecture Components

- **Main application (plugin loader)**
- **Plugin interface** (standard methods or hooks)
- **Plugin module** (external file with specific structure)

❏ Plugin Example

Plugin Interface (base class):

python

CopyEdit

```
class PluginInterface:
    def run(self):
        raise NotImplementedError("Plugin must implement run()")
```

Plugin 1 (plugin_hello.py):

python

CopyEdit


```
from plugin_interface import PluginInterface class HelloPlugin(PluginInterface): def run(self): print("Hello from Plugin!")
```

Main App:

python

CopyEdit

```
import importlib plugins = ['plugin_hello'] for p in plugins: module = importlib.import_module(p) plugin = module.HelloPlugin() plugin.run()
```

🔗 Real-World Python Plugin Tools

- **Burp Suite Extensions via Jython**
- **Ghidra Plugins using Python**

- IDA Pro scripts
- Nmap NSE scripts